

PART I

- 在 "Micrium/Software/uCOS-II/Source/os_time.c" 的 OSTimeDly() 中修改讓 delay = 0 時可以讓出 CPU 進行排程。

```
71     if (ticks >= 0u) {                                     /* 0 means no delay! */
72         OS_ENTER_CRITICAL();
73         y = OSTCBCur->OSTCBY;                               /* Delay current task */
74         OSRdyTbl[y] &= (OS_PRIO)~OSTCBCur->OSTCBBitX;
75         OS_TRACE_TASK_SUSPENDED(OSTCBCur);
76         if (OSRdyTbl[y] == 0u) {
77             OSRdyGrp &= (OS_PRIO)~OSTCBCur->OSTCBBitY;
78         }
79         OSTCBCur->OSTCBDly = ticks;                           /* Load ticks in TCB */
80         OS_TRACE_TASK_DLY(ticks);
81         OS_EXIT_CRITICAL();
82         OS_Sched();                                           /* Find next task to run! */
83     }
84 }
```

- 在 "Micrium/Software/uCOS-II/Source/os_core.c" 的 OS_SchedNew() 中實作了 EDF，首先過濾非就緒狀態或未參與排程的任務，僅比較可執行且具截止時間的任務遍歷所有已到達執行時間的任務，找出截止時間最早的任務並指派為最高可執行優先權；若 deadline 相同則選 TaskID 較小者，若仍相同則優先讓目前執行中的任務繼續運行，以避免不必要的上下文切換。

```
1945 #if ALGORITHM == EDF
1946     OS_TCB* ptcB = OSTCBLst;
1947     int timeTag = OSTimeGet();
1948     int earlyDeadline = 65535;
1949     int highPrio = OS_TASK_IDLE_PRIO;
1950     int highID = 63;
1951
1952     while (ptcB != NULL && ptcB->OSTCBPrio != OS_TASK_IDLE_PRIO)
1953     {
1954         if (ptcB->OSTCBStat != OS_STAT_RDY || ptcB->OSTCBExtPtr == NULL) {
1955             ptcB = ptcB->OSTCBNext;
1956             continue;
1957         }
1958         if (timeTag >= ((task_para_set*)(ptcB->OSTCBExtPtr))->TaskArriveTime) {
1959             if (((task_para_set*)(ptcB->OSTCBExtPtr))->TaskDeadLine < earlyDeadline) {
1960                 earlyDeadline = ((task_para_set*)(ptcB->OSTCBExtPtr))->TaskDeadLine;
1961                 highID = ((task_para_set*)(ptcB->OSTCBExtPtr))->TaskID;
1962                 highPrio = ptcB->OSTCBPrio;
1963             }
1964             else if (((task_para_set*)(ptcB->OSTCBExtPtr))->TaskDeadLine == earlyDeadline) {
1965                 if (((task_para_set*)(ptcB->OSTCBExtPtr))->TaskID < highID) {
1966                     earlyDeadline = ((task_para_set*)(ptcB->OSTCBExtPtr))->TaskDeadLine;
1967                     highID = ((task_para_set*)(ptcB->OSTCBExtPtr))->TaskID;
1968                     highPrio = ptcB->OSTCBPrio;
1969                 }
1970                 else if (((task_para_set*)(ptcB->OSTCBExtPtr))->TaskID == highID && ptcB == OSTCBCur) {
1971                     earlyDeadline = ((task_para_set*)(ptcB->OSTCBExtPtr))->TaskDeadLine;
1972                     highID = ((task_para_set*)(ptcB->OSTCBExtPtr))->TaskID;
1973                     highPrio = ptcB->OSTCBPrio;
1974                 }
1975             }
1976         }
1977         ptcB = ptcB->OSTCBNext;
1978     }
1979     OSPrioHighRdy = highPrio;
1980 #endif
1981 }
```

PART II

- 在" Micrium/Software/uCOS-II/Source/ucos_ii.h "新增 aperiodic_task_para_set 用於存非週期性任務；新增 APERIODIC_TASK_NUMBER 存非週期性任務數量；新增 InputAperiodicjobsFile()函式讀取非週期性任務檔案。

```
94  typedef struct aperiodic_task_para_set {
95      INT32U TaskID;
96      INT32U TaskArriveTime;
97      INT32U TaskExecutionTime;
98      INT32U TaskAbsoluteDeadline;
99      INT32U TaskFinishedTime;
100     INT16U TaskRemainTime;
101     INT8U  TaskIsArrived;
102     INT8U  TaskIsFinished;
103 } aperiodic_task_para_set;
104
105 int TASK_NUMBER;
106 int APERIODIC_TASK_NUMBER;
107 OS_STK** Task_STK;
108 task_para_set TaskParameter[OS_MAX_TASKS];
109 aperiodic_task_para_set AperiodicTaskParameter[OS_MAX_TASKS];
110 FILE* fp;
111 FILE* Output_fp;
112 errno_t Output_err;
113
114 void OutputInit(void);
115 void InputFile(void);
116 void InputAperiodicjobsFile(void);
```

- 在"Microsoft/Windows/Kernel/OS2/app_hooks.c"
 - ✧ InputFile():如果輸入沒有第三項，代表是 CUS，並進行初始化設定。

```
111     else if (i == 1) {
112         TaskParameter[j].TaskArriveTime = TaskInfo[i];
113         if (ptr == NULL) {
114             CUS_SERVER_SIZE = (float)TaskInfo[i] / 100.0;
115             TaskParameter[j].TaskArriveTime = 0;
116             TaskParameter[j].TaskPriority = 0;
117             TaskParameter[j].TaskDeadline = 65535;
118         }
119     }
```

- ✧ InputAperiodicjobsFile():改寫自 InputFile()，用於讀入非週期性任務。

```
135 void InputAperiodicjobsFile() {
136     errno_t err;
137
138     if ((err = fopen_s(&fp, APERIODIC_FILE_NAME, "r")) == 0) printf("\nThe file %s was opened\n\n", APERIODIC_FILE_NAME);
139     else printf("\nThe file %s was not opened\n\n", APERIODIC_FILE_NAME);
140
141     char str[MAX];
142     char* ptr;
143     char* pTmp = NULL;
144     int TaskInfo[INFO], i, j=0;
145     APERIODIC_TASK_NUMBER = 0;
146
147     while (fgets(str, sizeof(str) - 1, fp) != NULL) {
148         i = 0;
149         ptr = strtok_s(str, " ", &pTmp);
150
151         while (ptr != NULL) {
152             TaskInfo[i] = atoi(ptr);
153             ptr = strtok_s(NULL, " ", &pTmp);
154
155             if (i == 0) {
156                 AperiodicTaskParameter[j].TaskID = TaskInfo[0];
157                 AperiodicTaskParameter[j].TaskIsArrived = 0;
158                 AperiodicTaskParameter[j].TaskIsFinished = 0;
159             }
160             else if (i == 1) AperiodicTaskParameter[j].TaskArriveTime = TaskInfo[i];
161             else if (i == 2) {
162                 AperiodicTaskParameter[j].TaskExecutionTime = TaskInfo[i];
163                 AperiodicTaskParameter[j].TaskRemainTime = TaskInfo[i];
164             }
165             else if (i == 3) AperiodicTaskParameter[j].TaskAbsoluteDeadline = TaskInfo[i];
166             i++;
167         }
168         if (i > 0) {
169             APERIODIC_TASK_NUMBER++;
170             j++;
171         }
172     }
173     fclose(fp);
174     printf("Total Aperiodic Jobs Loaded: %d\n", APERIODIC_TASK_NUMBER);
175 }
```

- ✧ App_TimeTickHook():程式用於處理 CUS 對非週期性任務的排程：每次時間到達時計算新到達的任務，若目前沒有執行中的非週期性任務且新的任務能在其絕對截止時間內完成，則更新 CUS 截止時間並開始執行該任務；若已有任務執行中或新的任務無法如期完成，則僅標示等待或直接拒絕；當伺服器截止時間抵達時，會嘗試切換到下一個可接受的非週期性任務並更新截止時間，確保在不影響週期任務的前提下處理非週期性請求。

```

401 if (CUS_SERVER_SIZE > 0) {
402     int timeTag = OSTimeGet() + 1;
403     for (int i = 0; i < APERIODIC_TASK_NUMBER; i++) {
404         if (AperiodicTaskParameter[i].TaskArriveTime == timeTag) {
405             AperiodicTaskParameter[i].TaskIsArrived = 1;
406
407             if (timeTag < ServerDeadline && CurrentJobIndex != -1) {
408                 LOG_print(3, "./Output.txt", "%2d Aperiodic job (%d) arrives. Do nothing.\n",
409                     timeTag,
410                     AperiodicTaskParameter[i].TaskID
411                 );
412             }
413             else if (CurrentJobIndex == -1) {
414                 int newDeadline = timeTag + (INT32U)(AperiodicTaskParameter[i].TaskExecutionTime / CUS_SERVER_SIZE);
415
416                 if (AperiodicTaskParameter[i].TaskAbsoluteDeadline >= newDeadline) {
417                     LOG_print(3, "./Output.txt", "%2d Aperiodic job (%d) arrives and sets CUS's deadline as %d.\n",
418                         timeTag,
419                         AperiodicTaskParameter[i].TaskID,
420                         newDeadline
421                     );
422
423                     OS_ENTER_CRITICAL();
424                     ServerDeadline = newDeadline;
425                     CurrentJobIndex = i;
426
427                     if (CusTCB != NULL) {
428                         if (((task_para_set*)(CusTCB->OSTCBExtPtr)) != NULL) ((task_para_set*)(CusTCB->OSTCBExtPtr))->TaskDeadline = ServerDeadline;
429                         OSTaskResume(CusTCB->OSTCBPrio);
430                     }
431                     OS_EXIT_CRITICAL();
432                 }
433                 else {
434                     LOG_print(3, "./Output.txt", "%d\tAperiodic job (%d) rejects scheduling.\n",
435                         timeTag,
436                         AperiodicTaskParameter[i].TaskID
437                     );
438                     AperiodicTaskParameter[i].TaskIsFinished = 1;
439                 }
440             }
441         }
442     }
443 }
444
445 if (timeTag == ServerDeadline) {
446     int nextTaskIdx = -1;
447     for (int i = 0; i < APERIODIC_TASK_NUMBER; i++) {
448         if (AperiodicTaskParameter[i].TaskIsArrived && !AperiodicTaskParameter[i].TaskIsFinished && i != CurrentJobIndex) {
449             nextTaskIdx = i;
450             break;
451         }
452     }
453
454     if (nextTaskIdx != -1) {
455         int new_deadline = timeTag + (int)(AperiodicTaskParameter[nextTaskIdx].TaskExecutionTime / CUS_SERVER_SIZE);
456
457         if (AperiodicTaskParameter[nextTaskIdx].TaskAbsoluteDeadline >= new_deadline) {
458             LOG_print(3, "./Output.txt", "%2d Aperiodic job (%d) sets CUS's deadline as %d.\n",
459                 timeTag,
460                 AperiodicTaskParameter[nextTaskIdx].TaskID,
461                 new_deadline
462             );
463
464             OS_ENTER_CRITICAL();
465             ServerDeadline = new_deadline;
466             CurrentJobIndex = nextTaskIdx;
467
468             if (CusTCB != NULL) {
469                 task_para_set* cus_ext = (task_para_set*)CusTCB->OSTCBExtPtr;
470                 if (cus_ext != NULL) {
471                     cus_ext->TaskDeadline = ServerDeadline;
472                 }
473                 OSTaskResume(CusTCB->OSTCBPrio);
474             }
475             OS_EXIT_CRITICAL();
476         }
477         else {
478             LOG_print(3, "./Output.txt", "%2d Aperiodic job (%d) rejects scheduling.\n",
479                 timeTag,
480                 AperiodicTaskParameter[nextTaskIdx].TaskID
481             );
482             AperiodicTaskParameter[nextTaskIdx].TaskIsFinished = 1;
483         }
484     }
485 }

```

➤ 在 "Microsoft/Windows/Kernel/OS2/main.c"

✧ 宣告 CUS 任務 `cus()`、server size、目前 server 的 deadline 與 state、當前執行中的非週期任務索引與指標，以及對應的 TCB，用來管理 CUS Server 的執行與非週期任務調度。

```
90     static void task(void* p_arg);
91     static void cus(void* p_arg);
92
93     int missDeadline = 0;
94     int missTask=0;
95
96     float CUS_SERVER_SIZE;
97     int ServerDeadline=0;
98     int ServerState=0;
99     int CurrentJobIndex=-1;
100
101     aperiodic_task_para_set* CurrentCusJob= NULL;
102     OS_TCB* CusTCB= NULL;
```

✧ `cus()`: 負責在系統中沒有週期任務需要執行時運行非週期請求：當沒有待執行的非週期性任務時會等待並讓出 CPU；一旦有任務可執行，便以 server size 逐步扣減剩餘執行時間。當任務完成後，會記錄完成資訊、標記為已結束並將 `CurrentJobIndex` 重置，等待系統排程下一個合適的作業。

```
148 void cus(void* p_arg) {
149     while (1) {
150         OS_ENTER_CRITICAL();
151         if (CurrentJobIndex == -1) {
152             OS_EXIT_CRITICAL();
153             OSTaskSuspend(OS_PRIO_SELF);
154
155             OS_ENTER_CRITICAL();
156             if (CurrentJobIndex == -1) {
157                 OS_EXIT_CRITICAL();
158                 continue;
159             }
160         }
161         OS_EXIT_CRITICAL();
162
163         if (CurrentJobIndex < 0 || CurrentJobIndex >= APERIODIC_TASK_NUMBER) continue;
164
165         aperiodic_task_para_set* curTask = &AperiodicTaskParameter[CurrentJobIndex];
166
167         while (curTask->TaskRemainTime > 0) {
168             LOG_print(3, "/Output.txt", "%2d Aperiodic job(%d) is running\n", OSTimeGet(), curTask->TaskID);
169
170             int timeTag = OSTimeGet();
171             while (OSTimeGet() == timeTag) {}
172             curTask->TaskRemainTime--;
173         }
174
175         OS_ENTER_CRITICAL();
176         if (curTask->TaskRemainTime == 0) {
177             int timeTag = OSTimeGet();
178
179             OS_TCB* ptcb = OSTCBLIST;
180             OS_TCB* next_tcb = NULL;
181             INT32U min_deadline = 0xFFFFFFFF;
182             INT16U min_task_id = 0xFFFF;
183
184             while (ptcb != NULL) {
185                 if (ptcb->OSTCBStat == OS_STAT_RDY &&
186                     ptcb->OSTCBBody == 0 &&
187                     ptcb != OSTCBCur &&
188                     ptcb->OSTCBPrio != OS_TASK_IDLE_PRIO)
189                 {
190                     if (ptcb->OSTCBExtPtr != NULL) {
191                         if (((task_para_set*)(ptcb->OSTCBExtPtr))>TaskDeadline < min_deadline) {
192                             min_deadline = ((task_para_set*)(ptcb->OSTCBExtPtr))>TaskDeadline;
193                             min_task_id = ((task_para_set*)(ptcb->OSTCBExtPtr))>TaskID;
194                             next_tcb = ptcb;
195                         }
196                         else if (((task_para_set*)(ptcb->OSTCBExtPtr))>TaskDeadline == min_deadline) {
197                             if (((task_para_set*)(ptcb->OSTCBExtPtr))>TaskID < min_task_id) {
198                                 min_task_id = ((task_para_set*)(ptcb->OSTCBExtPtr))>TaskID;
199                                 next_tcb = ptcb;
200                             }
201                         }
202                     }
203                 }
204                 ptcb = ptcb->OSTCBNext;
205             }
206         }
207     }
208 }
```

```

206
207     int next_id = 0;
208     int next_job_no = 0;
209
210     if (next_tcb != NULL) {
211         task_para_set* next_ext = (task_para_set*)next_tcb->OSTCBExtPtr;
212         next_id = next_ext->TaskID;
213         next_job_no = next_ext->TaskID;
214     }
215     else {
216         next_id = 43;
217         next_job_no = 0;
218     }
219
220     LOG_print(3, "../Output.txt", "N2d Aperiodic job (N6) is finished.\n", timeTag, curTask->TaskID);
221     LOG_print(3, "../Output.txt", "N2d Completion\ttask(N2d)(N2d)\ttask(N2d)(N2d)\tN6\tN6\tH/A\n",
222         timeTag,
223         TaskParameter[TASK_NUMBER - 1].TaskID,
224         curTask->TaskID,
225         next_id,
226         next_job_no,
227         timeTag - curTask->TaskArriveTime,
228         (timeTag - curTask->TaskArriveTime) - curTask->TaskExecutionTime
229     );
230
231     curTask->TaskIsFinished = 1;
232     CurrentJobIndex = -1;
233     task_para_set* task_data = OSTCBCur->OSTCBExtPtr;
234     task_data->TaskID++;
235
236     }
237     OS_EXIT_CRITICAL();
238 }
239

```

- ✧ main():若 CUS_SERVER_SIZE > 0，先為所有週期性任務分配 stack 並建立任務，接著為 CUS Server 任務單獨分配 stack 並建立任務，最後將 CUS Server 的 TCB 指標存入 CusTCB 以便後續調度；若 CUS_SERVER_SIZE = 0，則只建立週期任務而不建立 CUS Server 任務。

```

300
301     if (CUS_SERVER_SIZE > 0) {
302         for (int n = 0; n < TASK_NUMBER - 1; n++)
303         {
304             Task_STK[n] = malloc(TASK_STACKSIZE * sizeof(int));
305             OSTaskCreateExt(task,
306                 &TaskParameter[n],
307                 &Task_STK[n][TASK_STACKSIZE - 1],
308                 TaskParameter[n].TaskPriority,
309                 TaskParameter[n].TaskID,
310                 &Task_STK[n][0],
311                 TASK_STACKSIZE,
312                 &TaskParameter[n],
313                 (OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR));
314         }
315
316         int cusIdx = TASK_NUMBER - 1;
317         Task_STK[cusIdx] = malloc(TASK_STACKSIZE * sizeof(int));
318
319         OSTaskCreateExt(cus,
320             &TaskParameter[cusIdx],
321             &Task_STK[cusIdx][TASK_STACKSIZE - 1],
322             TaskParameter[cusIdx].TaskPriority,
323             TaskParameter[cusIdx].TaskID,
324             &Task_STK[cusIdx][0],
325             TASK_STACKSIZE,
326             &TaskParameter[cusIdx],
327             (OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR));
328
329         CusTCB = OSTCBPrioTbl[TaskParameter[cusIdx].TaskPriority];
330
331         if (CusTCB == NULL) printf("Error: CUS Task TCB not found!\n");
332     }
333     else {
334         for (int n = 0; n < TASK_NUMBER; n++)
335         {
336             Task_STK[n] = malloc(TASK_STACKSIZE * sizeof(int));
337             OSTaskCreateExt(task,
338                 &TaskParameter[n],
339                 &Task_STK[n][TASK_STACKSIZE - 1],
340                 TaskParameter[n].TaskPriority,
341                 TaskParameter[n].TaskID,
342                 &Task_STK[n][0],
343                 TASK_STACKSIZE,
344                 &TaskParameter[n],
345                 (OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR));
346         }
347     }
348 }

```